Projet GL — Documentation de validation
Gestion des risques et gestion des rendus

Groupe 53 : *Les misérables*

07-Janvier-2026

1 Gestion des risques

1.1 Objectif

Un projet logiciel comporte inévitablement des erreurs (interprétation, implémentation, intégration). L'objectif de cette section est de **prioriser les dangers critiques** (impact élevé sur la qualité du projet) et de définir des **actions concrètes et vérifiables** (réponse oui/non) permettant de réduire la probabilité d'occurrence et l'impact de ces risques.

Rappel : certains bugs peuvent être rares mais coûteux (compilation cassée). Nous privilégions donc les risques à **impact fort**, même si leur correction est "simple".

1.2 Table d'analyse des risques

ID	Risque / danger	Impact	Proba.	Actions (concrètes, vérifiables)
R1	Oubli d'une date de suivi ou rendu	Fort	Moyen	• Mettre les 4 dates dans un agenda partagé + rappels • Pinner un message "Deadlines" dans le canal du groupe.
R2	Dépôt partagé non compilable (oubli `git add`, conflit, commit cassé).	Très fort	Moyen	• Interdire le commit direct sur `main` • Exiger `mvn clean` avant merge. • Vérif : on refait un clone dans un dossier vide et ça compile.
R3	Infrastructure de tests incomplète ou trompeuse (faux positifs).	Fort	Moyen	• Ajouter 3 tests "sanity" (valid+invalid) par passe (syntaxe, contexte). • Exécuter le script `common-tests.sh` avant rendu.
R4	Régression : une correction casse une fonctionnalité déjà validée.	Fort	Moyen	• MR/PR obligatoire + relecture par 1 personne. • Vérif : le test reproduisant le bug est ajouté au dépôt.

ID	Risque / danger	Impact	Proba.	Actions (concrètes, vérifiables)
R5	Mauvaise interprétation de la grammaire	Moyen	Moyen	• Identifier règles sensibles (priorités opérateurs, assign, conversions). • Créer des tests ciblés (petits) + 1 test "mixte" (long).
R6	Problème environnement (Java/Maven/ANTLR/IMA)	Fort	Moyen	• Vérifier versions : `java -version`, `mvn -v`. • Vérifier IMA : `ima -v` sur la machine de rendu. • Vérif : exécution d'un exemple minimal end-to-end (compile + test).
R7	Répartition du travail déséquilibrée / blocage d'une personne.	Moyen	Moyen	• Pair-programming si blocage > 1h. • Vérif : chaque tâche a un owner + une date de fin.
R8	Rendu incomplet : documentation manquante / non à jour.	Fort	Moyen	• Gabarit doc de validation + sections obligatoires • Relecture doc la veille du suivi. • Vérif : PDF généré depuis le dépôt

1.3 Risques prioritaires

Nous considérons comme **prioritaires** les risques à impact très fort (R1, R2, R3, R6) car ils peuvent rendre le compilateur inutilisable ou décrédibiliser le projet (dépôt cassé, tests non fiables, environnement non opérationnel).

2 Gestion des rendus (Release management process)

2.1 Objectif

Même avec des tests automatisés, certains contrôles ne sont pas totalement automatisables (ex : vérifier que le dépôt partagé fonctionne réellement, ou vérifier l'environnement). Nous définissons donc une **checklist de rendu** : une suite d'actions simples, vérifiables, à exécuter avant chaque suivi / rendu.

2.2 Checklist avant chaque suivi / rendu

A. Vérification dépôt / état Git

- ☐ `git status` : aucun fichier critique non committé. *(évite l'oubli de fichiers)*
- ☐ `git pull` sur la branche cible et résolution propre des conflits. *(évite divergence)*

B. Build + tests automatiques obligatoires

- ☐ `mvn -q clean test` *(compile + exécute tests + common-tests.sh)*

- ☐ Lancer au moins 1 test “sanity” par passe :
  - ☐ syntaxe valid + invalid
  - ☐ contexte valid + invalid

C. Vérification environnement (si codegen/IMA concerné)

- ☐ `ima -v` fonctionne dans un nouveau terminal. *(vérifie PATH + binaire)*
- ☐ Compilation + exécution d’un exemple minimal end-to-end (compile + test). *(preuve que tout marche)*

D. Vérification “dépôt partagé” (reproductibilité)

- ☐ Refaire un clone dans un répertoire vide (ou sur une autre machine). *(pas seulement votre clone local)*
- ☐ Refaire : `mvn -q clean test`. *(garantit le rendu utilisable par l’enseignant)*

E. Documentation / livrables

- ☐ README à jour (setup + commandes test). *(rendu exploitable)*
- ☐ Doc de validation à jour (incluant cette section risques + checklist). *(conforme consignes)*

2.3 Quand exécuter cette checklist ?

- **Avant chaque suivi**
- **La veille + le matin du rendu final** (23/01 soutenance)
- **Avant toute fusion (MR/PR)** : au minimum parties A et B